

GLUCOEDGE: An Engineering Study of On-Device Five-Class Glucose Trend Forecasting and INT8 Quantization

Mohamed Menasy
Independent Researcher
mohamedmenasy@gmail.com

October 2026

Abstract

We study how INT8 quantization changes a compact glucose trend classifier and assess whether limited calibration coverage could explain the observed errors. GLUCOEDGE is an open-source Android and iOS prototype that uses 12 five-minute continuous glucose monitor (CGM) readings to predict one of five rate-of-change classes over the next 15 minutes. A 2,909-parameter convolutional network was trained with class-weighted cross-entropy on public Weinstock data prepared by GlucoBench. On 127,165 aggregate test windows, float LiteRT preserved the checkpoint’s reported accuracy (0.5184) and macro recall (0.5060). Static INT8 quantization using the first 200 validation windows reduced the artifact from 16.95 to 11.70 KiB (31.0%). Accuracy rose to 0.5866, but macro recall fell to 0.4135: stable-class recall improved while all four directional recalls declined. Their mean fell from 0.5019 to 0.3537. A provenance audit shows that the 200 calibration windows came from one participant’s 17.5 hours of data and that the resulting INT8 input range ends at 239.9 mg/dL, below readings in 24.4% of training windows. Clipping is therefore a plausible source of error, but its contribution has not been isolated by comparing calibration selections. On a Galaxy S22 Ultra, timed inference-path mean/p95 latencies were 0.334/0.540 ms for float ($n = 100$) and 0.484/0.665 ms for INT8 ($n = 21$), excluding softmax and class selection. On an iPhone 18 Pro Max with equal samples ($n = 100$ per model, two rounds), mean latencies were 0.062–0.073 ms for float and 0.101–0.110 ms for INT8, and the same golden vectors passed. These uncontrolled observations show no INT8 latency benefit on either phone. Context-prefix overlap (13.6% of test windows score targets in validation rows; none score training rows) and test-informed class selection also limit the predictive conclusions. These results describe the converted models; they do not show whether broader calibration coverage would recover minority-class recall. GLUCOEDGE is a research demonstration, not a medical device or treatment system.

1 Introduction

A CGM sampled every five minutes produces 12 observations per hour. These readings can be used to predict a near-future glucose value, event, or trend [1]. Here, we study such predictions for research purposes; they are not intended to guide treatment.

These targets define different learning problems. Value regression estimates a future glucose level, event prediction asks whether a glycemic event will occur, and trend classification assigns a directional rate category. Published studies also differ in prediction horizon, use of contextual inputs, cohort, split construction, and metric. Reviews and comparison indices document these differences. Public benchmarks and cross-dataset experiments help distinguish performance within a dataset from transfer to another data source [1–4]. Scores from incompatible protocols should therefore not be compared directly.

Commercial CGM trend arrows are related but not equivalent targets. They summarize recent or current glucose direction and rate using device-specific definitions [5, 6]. In contrast, GLUCOEDGE derives each label from the change between the current sample and the sample 15 minutes later. The arrow-like rate bands provide a compact research representation, but the future-derived label neither reproduces a manufacturer’s algorithm nor provides a dosing rule.

Deployment raises further questions. Conversion may change a model’s outputs, and the client may need different input handling. Lower precision also does not guarantee lower latency on a given device. Class imbalance can hide conversion errors: accuracy can increase as predictions concentrate on a majority class even while minority-class recall declines [7].

The motivating research question is: *How does calibration-data coverage affect minority-class performance and deployment cost in compact glucose trend classifiers?* Here, coverage refers to the input values and trajectories represented during quantizer calibration, not calibration of predicted probabilities. We have results from one trained checkpoint, one calibration selection, matched float/INT8 evaluations, and one phone, plus an audit of window provenance and calibration-set compo-

sition. These results show the observed trade-off and where the input-range limit came from. They cannot establish what would change if calibration coverage alone were varied.

This paper follows the model from public CGM preprocessing through conversion to offline replay on Android and iOS. It reports:

- a segment-aware construction of 12-sample CGM windows and future-derived, five-class 15-minute trend labels;
- matched evaluation of PyTorch, float LiteRT, and static INT8 LiteRT artifacts, showing the opposing changes in accuracy and directional recall on the same aggregate test frame;
- cross-runtime golden-vector tests and physical-device latency observations for Android (Kotlin) and iOS (Swift) `CompiledModel` implementations that run the same model files; and
- a provenance audit of window splits and calibration inputs, which traces the INT8 input range to a single participant’s data, and the comparisons needed to test whether that range explains the recall loss.

All source code, tests, synthetic replay data, and shipped model artifacts are available in the project repository.¹

2 Related Work

2.1 Forecasting tasks, horizons, and inputs

CGM forecasting has expanded from autoregressive value prediction using sensor history [8] to recurrent and attention models trained with CGM and contextual variables [9]. Recurrent models have also been used to estimate predictive variance rather than only a point value [10], while convolutional–recurrent models fuse CGM, insulin, and meal sequences before predicting 30- or 60-minute values [11]. Architecture and input modality change together across these studies, so their results do not isolate the effect of model family alone. Cross-dataset evaluation asks a different question: whether a trained model generalizes when the source dataset, cohort, and collection process change [4].

Table 1 summarizes these design differences. It distinguishes offline predictive evaluation from reported device execution and omits scores that would be misleading to compare across protocols.

2.2 Benchmarks and evaluation protocols

The split determines what a study can claim about generalization. Subject-dependent splits evaluate people represented during training; held-out-participant splits evaluate unseen people within the same dataset. Cross-dataset evaluation also changes the cohort and data pipeline. GlucoBench standardizes preprocessing, splits, and baselines across five public datasets [1]; experiments across four datasets directly test changes in data source [4].

GLUCOEDGE uses GlucoBench’s largest type-1 diabetes cohort, referred to as Weinstock. Its source study collected blinded CGM from older adults with long-standing type-1 diabetes [12].

Time-series splitting also requires care because nearby and overlapping windows are dependent. The validity of cross-validation for autoregressive prediction depends on the residual structure rather than a generic random-split rule [13], and leakage can produce answers to a less demanding problem than the one a study claims to test [14]. Even with a fixed split, training and implementation choices contribute benchmark variance [15]. Results therefore need to state the participant, temporal, and dataset boundaries used in evaluation.

2.3 Trend representations and class imbalance

Commercial trend arrows summarize recent or current movement using device-specific categories whose rate bands and look-back intervals differ [5, 6]. A future label instead asks whether the trajectory after the input window falls, remains stable, or rises. GLUCOEDGE uses these rate bands to define a research target from the sample 15 minutes after the window. It does not reproduce a commercial arrow algorithm or support treatment decisions.

In the Weinstock windows used here, most labels are stable and faster changes are relatively rare. Overall accuracy therefore weights the stable class in proportion to its prevalence. Macro-averaged recall, equivalent to balanced accuracy for this single-label setting, gives each class equal weight and makes minority-class degradation visible [7]. Macro recall still does not establish event-level or clinical performance.

2.4 Edge inference and quantization

One-dimensional CNNs extract local temporal features with compact feed-forward networks [16].

Mobile inference systems can reduce neural-network execution cost through compression and runtime-specific scheduling [17], but latency is a property of the model, runtime, and hardware together. Li et al. reported Android timing for a glucose forecasting model [11]; broader

¹<https://github.com/mohamedmenasy/GlucoEdge>

Table 1: Representative CGM forecasting studies and evaluation scope. Metrics are not compared across rows because datasets and protocols differ.

Study	Target	Horizon	Inputs	Evaluation scope	Device evidence
Reifman et al. [8]	Value	30 min	CGM	9 participants	None reported
Mirshekarian et al. [9]	Value	30/60 min	CGM + context	OhioT1DM/synthetic	None reported
Martinsson et al. [10]	Value + variance	30/60 min	CGM	OhioT1DM	None reported
Li et al. [11]	Value	30/60 min	CGM, insulin, meals	10 real + 10 simulated	Android timing
Glucobench [1]	Value trajectory	60 min	Dataset-dependent	Five public datasets	None reported
Ghimire et al. [4]	Value	30/60 min	CGM	Four datasets	None reported
GLUCOEDGE (this work)	Trend class	15 min	CGM	Weinstock aggregate test	Android/iOS replay timing

smartphone and tiny-device benchmarks also measure specified workloads and platforms because model size alone does not determine speed [18, 19].

Integer quantization approximates floating-point tensors with discrete values parameterized by a scale and zero point [20]. Static post-training quantization estimates activation ranges from calibration inputs before conversion [21, 22]. Low-bit studies show that clipping, range estimation, and rounding choices can determine the resulting error [23, 24]; specialized methods have also been proposed to improve post-training accuracy when calibration sets are small [25]. These findings support checking converted outputs and per-class metrics alongside model size.

GLUCOEDGE connects public preprocessing with conversion parity, class-wise evaluation, and offline replay on one Android and one iOS device; it does not establish live-sensor, clinical, or cross-hardware performance.

3 Problem Formulation

Let g_t be the CGM value at time t , in mg/dL, with observations every five minutes. The input is

$$\mathbf{x}_t = [g_{t-55}, g_{t-50}, \dots, g_t] \in \mathbb{R}^{12}. \quad (1)$$

The future rate of change is computed from a point outside the input window:

$$r_t = \frac{g_{t+15} - g_t}{15} \quad [\text{mg/dL/min}]. \quad (2)$$

The class y_t is assigned by

$$y_t = \begin{cases} \text{falling_fast}, & r_t \leq -2, \\ \text{falling}, & -2 < r_t \leq -1, \\ \text{stable}, & -1 < r_t < 1, \\ \text{rising}, & 1 \leq r_t < 2, \\ \text{rising_fast}, & r_t \geq 2. \end{cases} \quad (3)$$

The 12 readings span 55 elapsed minutes; “one-hour history” refers to 12 samples at a five-minute cadence. The history length and 15-minute horizon are fixed engineering choices for this prototype. The five-class target is a simplification based on two samples and does not describe the full intervening trajectory. The boundaries

use rate bands commonly represented by CGM trend arrows, while collapsing device-specific arrow taxonomies into five classes [5, 6]. No threshold was optimized on the test labels.

4 Data and Methods

4.1 Dataset and preprocessing

The experiments use the Weinstock data distributed through GlucoBench. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12]. GlucoBench reports 200 participants before preprocessing and 192 afterwards, with a five-minute sampling interval [1]; our audit of the formatter output reproduces both counts. The configuration used here disables scaling and retains glucose in raw mg/dL. The formatter rounds timestamps to the minute, splits each participant’s record at gaps longer than 45 minutes, drops segments with fewer than 240 original readings, and linearly interpolates each remaining segment onto a five-minute grid that starts at its first reading. Each segment is tagged with an `id_segment`. Of the 579,782 grid values, 75.8% coincide with an original reading. A further 19.8% fall between two consecutive readings no more than 7.5 minutes apart, because reading times drift relative to the grid; 4.4% fill one or more missing readings. Because interpolation precedes windowing, interpolated values may occur among the inputs or at samples used to form targets. The configured split holds out 10% of participants (20 people) for an out-of-distribution component and forms temporal validation and test components from the remaining participants. The formatter combines the test components into one frame. We report metrics for that aggregate without separate in- and out-of-distribution results.

Windows are generated independently within each (`id`, `id_segment`) group and ordered by time. A window is emitted only when all 12 input points and the three following horizon steps fit in the same segment. The stride is one reading, matching an application that updates every five minutes. This rule prevents a sensor gap or participant boundary from being treated as a continuous trajectory. Adjacent stride-one windows overlap heavily and are statistically dependent. The resulting

Table 2: Window distribution after GlucoBench formatting. Percentages are within each split.

Class	Train	Aggregate test
falling_fast	13,260 (3.12%)	3,692 (2.90%)
falling	40,122 (9.45%)	11,387 (8.95%)
stable	314,257 (74.04%)	95,998 (75.49%)
rising	39,428 (9.29%)	11,012 (8.66%)
rising_fast	17,395 (4.10%)	5,076 (3.99%)
Total	424,462	127,165

frames yielded 424,462 training, 55,289 validation, and 127,165 aggregate test windows from 172, 127, and 186 participants, respectively. The test windows comprise 57,068 from the 20 held-out participants and 70,097 from temporal segments of 166 training participants.

Table 2 shows the resulting class imbalance. Both fast classes exceed 13,000 training windows. A preliminary 20-epoch run obtained test recalls of 0.5333 and 0.5380 for **falling_fast** and **rising_fast**, respectively. Neither met the predeclared condition for collapsing the classes: fewer than 200 training examples or recall below 0.30. The deployed model therefore retains all five classes. Test recall informed this five-versus-three decision, so it must be treated as an exploratory development choice rather than independent validation.

4.2 Model and training

The **TrendCNN** accepts tensors of shape $(B, 1, 12)$. It applies a one-dimensional convolution from 1 to 24 channels, a second convolution from 24 to 36 channels, ReLU after each layer, adaptive average pooling over time, and a 36-to-5 linear head. Both convolutions use kernel size 3, stride 1, and padding 1. The network contains 2,909 trainable parameters.

Training uses inverse-frequency class-weighted cross-entropy, batch size 64, and Adam with learning rate 10^{-3} [26]. The artifact used for conversion was trained for 20 epochs with PyTorch seed 0 [27]. Validation loss was observed after every epoch; there was no early stopping or test-time selection. Classification reports were computed with scikit-learn [28]. Because the test distribution is dominated by **stable**, macro-averaged recall is the primary aggregate metric. For single-label multiclass prediction it is the mean of the five per-class recalls and is equivalent to balanced accuracy [7].

5 Conversion and Deployment

The seed-0 checkpoint was converted with LiteRT Torch. Float conversion uses a fixed $(1, 1, 12)$ input signature. INT8 conversion uses PyTorch 2 Export (PT2E) with LiteRT Torch’s `get_symmetric_quantization_config` helper (`is_per_channel=True`, `is_dynamic=False`).

Despite the helper’s name, only the weights are quantized symmetrically, per channel. Activations, including the model input, use per-tensor affine quantization whose ranges a histogram observer selects to minimize quantization error [21]. The first 200 validation windows serve as calibration inputs. Selection follows the dataset’s participant/segment and time order, with no randomization or attempt to balance input-range coverage; Section 7.1 shows that all 200 windows come from one participant. The converted model’s input scale is 0.9408 mg/dL per code with zero point -128 , so its representable input range is 0–239.9 mg/dL. The range starts at zero because the observer always includes zero, which leaves the codes below the lowest CGM reading unused.

For input scale s and zero point z , both clients use the affine form

$$q = \text{clip}(\text{round}(x/s) + z, -128, 127), \quad \hat{x} = s(q - z). \quad (4)$$

Quantization parameters are read from the model metadata rather than hardcoded. All three clients divide in single precision and round exact half-way values to the even integer: Python with `numpy.round`, Kotlin with `kotlin.math.round`, and Swift with `rounded(.toNearestOrEven)` [29–31]. They therefore produce identical integer inputs for any glucose value. The July device tests ran an earlier Kotlin version that rounded ties toward positive infinity. The golden-vector inputs are given to 0.1 mg/dL, and none of those values is a tie at the shipped input scale, so the expected outputs are unchanged.

The Kotlin/Jetpack Compose application replays a bundled synthetic trace. A 12-reading ring buffer resets after a cadence gap, mirroring training-time segment boundaries. Every complete window is classified locally; the merged Android manifest is tested to ensure it does not request the **INTERNET** permission. The float artifact is the default and INT8 can be selected for comparison.

Real devices use LiteRT 2.1.0 **CompiledModel** on CPU [32]. On the tested Apple-Silicon-hosted Android emulator, native model creation terminated with an illegal-instruction fault during a CPU feature probe for the Arm Scalable Matrix Extension (SME). Because this fault cannot be caught by Kotlin, emulators are selected before runtime initialization and use the classic interpreter with XNNPACK disabled. This workaround applies to the tested environment; it does not establish performance across LiteRT configurations.

A SwiftUI iOS application mirrors the Android client. It bundles the same two artifact files and synthetic trace by reference to the Android asset directory, so both platforms ship byte-identical models. It runs them through LiteRT’s Swift **CompiledModel** API on CPU, using the LiteRT 2.3.0 release binaries. Replay, the gap-resetting window buffer, the model switch, and the latency statistics follow the Kotlin implementation. iOS has no equivalent of Android’s **INTERNET** permission. Instead, a source

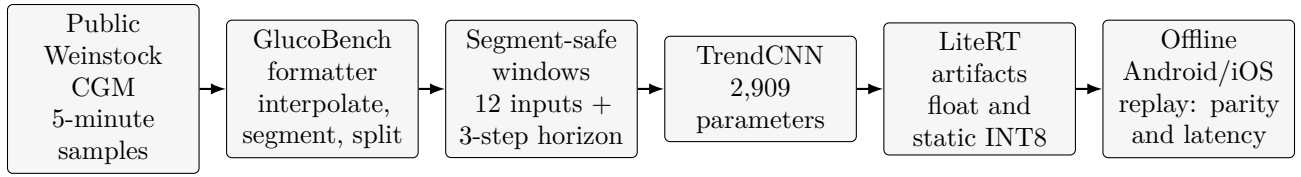


Figure 1: End-to-end GLUCOEDGE workflow. Patient data are used locally for training and evaluation; only model files and a synthetic replay trace are committed to the Android and iOS applications.

scan fails the build’s continuous-integration job if networking APIs appear in the application code. This guard is weaker than the Android manifest check.

6 Experimental Protocol

The aggregate test frame combines temporal and held-out-participant components. The implementation did not exclude context-prefix rows from eligible targets; Section 7.1 quantifies the resulting overlap, which involves validation rows but no training rows. The metrics therefore describe the shipped model under this protocol rather than separate temporal and participant-level generalization.

A provenance audit (`experiments/split_audit.py`) maps every window to the formatter rows it uses and assigns each row to the earliest split that lists it, because GlucoBench repeats rows from a preceding split as context. It reads row identities and timestamps for all splits but glucose values only from training and validation rows; it does not read labels, held-out glucose values, or predictions. It also records the composition of the calibration set and the INT8 model’s input quantization parameters.

The same 127,165 aggregate test windows and labels were passed through the PyTorch checkpoint, float LiteRT model, and INT8 LiteRT model. Accuracy, per-class recall, and macro recall were computed from each prediction set. A constant-`stable` classifier is included to show the effect of class prevalence. If n_c is class support, $N = \sum_c n_c$, and R_c is class recall, then

$$A = \sum_{c=1}^5 \frac{n_c}{N} R_c, \quad R_{\text{macro}} = \frac{1}{5} \sum_{c=1}^5 R_c. \quad (5)$$

We also summarize the four directional classes, defined here as all classes except `stable`, by their unweighted mean recall. This descriptive summary and the accuracy decomposition below use the existing rounded per-class reports. Neither required a new training or evaluation run. All differences refer to the same aggregate windows; no confidence intervals or significance tests were calculated.

File size is the exact serialized artifact size. Device latency is measured inside the application’s `classify` method with `System.nanoTime`; it includes buffer writes, input quantization when needed, model execution, and

output reading/dequantization. The timer stops before softmax, class selection, and construction of the returned prediction; model loading and UI updates are also outside its scope. Measurements were recorded during replay on a Samsung Galaxy S22 Ultra (SM-S908E, Android 16). The float statistics use the latest 100 inferences, while the recorded INT8 statistics contain 21 inferences after a model switch. The application reports p95 as the sorted sample at zero-based index $\lfloor 0.95(n-1) \rfloor$, so these estimates also have different rank resolution. The timings cover this portion of the inference path during replay. They measure neither the full classification call nor the kernel alone, and the runs were not controlled for a benchmark comparison.

The iOS client times the same region with the monotonic `DispatchTime` clock. Measurements were recorded on an iPhone 18 Pro Max (iPhone19,3, iOS 27.0.1) during $16\times$ replay of the synthetic trace, which produces one inference every 312.5 ms. Each model’s statistics use its latest 100 inferences, and the measurement was repeated in two rounds. The application code was a debug build; LiteRT itself is a prebuilt release binary. A separate test-runner benchmark calls each model 100 times back to back after one warm-up call, in three runs. It isolates the timed region from the interval between replay ticks. Calibration preparation time, peak memory, model-load time, and energy were not measured.

Cross-runtime parity was tested with 20 synthetic golden windows, four selected from each float-predicted class and including values above the INT8 input range. The test also verifies the SHA-256 hash of each bundled artifact. Float logits must match the Python output within 10^{-5} ; dequantized INT8 outputs are required to match bit-for-bit because both sides dequantize identical integer outputs with the same float32 operation. The iOS suite applies the same vectors and tolerances and also checks the class-name order. It runs on the physical iPhone, on the iOS simulator, and in continuous integration.

7 Results

7.1 Split and calibration provenance

Table 3 summarizes the audit. No test window has a target in a training row, and none repeats a training window. The context-prefix overlap instead affects 17,266

Table 3: Window provenance and calibration-set composition from the audit. Shares are within each split.

Window source	Windows	Share
Validation, target in validation rows	35,887	64.9%
Validation, copy of a training window	19,402	35.1%
Test, held-out participants	57,068	44.9%
Test, target in test rows	52,831	41.5%
Test, target in validation rows	17,266	13.6%
Test, target in training rows	0	0.0%
INT8 calibration set (first 200 validation windows)		
Participants; segments		1; 1
Distinct input readings; span	211; 17.5 h	
Copies of training windows	178 of 200	
Maximum input	240.0 mg/dL	
INT8 input range	0–239.9 mg/dL	

test windows (13.6%) whose targets fall in rows that GlucoBench assigns to validation; validation labels were not used for fitting or checkpoint selection. Most validation frames begin with 192 training rows of model context, so 19,402 validation windows (35.1%) repeat training windows exactly.

The calibration set is narrower than its size suggests. Its 200 overlapping windows come from one participant and one segment: 211 distinct readings spanning 17.5 hours, with 178 windows repeating training windows. The largest calibration input, 240.0 mg/dL, matches the upper end of the INT8 input range, 239.9 mg/dL. In development data, 24.4% of training windows and 26.3% of validation windows contain at least one reading above that range.

7.2 Predictive performance

Table 4 shows that float LiteRT preserved the checkpoint’s reported classification metrics. The constant majority-class baseline reaches 0.7549 accuracy, higher than either learned model, but its macro recall is only 0.2000. Accuracy alone would therefore favor a model that never predicts a directional class.

INT8 accuracy increased by 6.82 percentage points relative to float, while macro recall decreased by 9.25 points. Table 5 shows where the change occurred: recall improved only for **stable**. It fell for every minority class. The largest loss was in **falling_fast**, whose recall fell from 0.5119 to 0.1907, a 62.7% relative reduction. Mean directional recall fell from 0.5019 to 0.3537, a 14.82-percentage-point decrease derived from the rounded per-class values.

Weighting each recall change by its class support explains the accuracy gain. This calculation does not require predicted class frequencies. With the recorded supports, the stable-class recall gain contributes approximately +9.84 percentage points to accuracy; the four directional losses contribute −3.03 points in total. Sum-

Table 4: Aggregate test metrics. PyTorch and float LiteRT have identical reported values.

Model	Accuracy	Macro recall
Constant stable	0.7549	0.2000
PyTorch checkpoint	0.5184	0.5060
Float LiteRT	0.5184	0.5060
INT8 LiteRT	0.5866	0.4135

Table 5: Per-class recall before and after static INT8 quantization.

Class	Float	INT8	Change
falling_fast	0.5119	0.1907	−0.3212
falling	0.5484	0.4306	−0.1178
stable	0.5225	0.6529	+0.1304
rising	0.4761	0.3845	−0.0916
rising_fast	0.4712	0.4090	−0.0622
Directional mean	0.5019	0.3537	−0.1482

ming before rounding yields the reported +6.82-point change. The reported recalls are not enough to reconstruct the full confusion matrix or predicted class proportions.

7.3 Size, latency, and parity

Table 6 summarizes deployment measurements. INT8 reduced the artifact by 5,376 bytes, or 31.0%. Observed on-device latency did not decrease. The unequal Android sample counts prevent a formal comparison there. On the iPhone, with 100 inferences per model, INT8 had the higher mean and p95 latency in both rounds. Two uncontrolled rounds cannot establish a speed ordering, but neither phone shows an INT8 speed advantage.

The iPhone’s back-to-back benchmark measured mean latencies of 2.83–2.85 μ s for float and 3.69–3.72 μ s for INT8 across three runs, again with INT8 slower. These values are roughly 20–30 times lower than the in-app means for the same code. Most of the in-app time therefore comes from calling the model once per replay tick rather than from its arithmetic.

The July conversion record² reports a test maximum of 401 mg/dL and saturation in approximately 28% of test windows. We did not recompute this figure, so that held-out glucose values stay unused until a locked re-evaluation; Section 7.1 gives the corresponding development-data rates and the origin of the ceiling at 239.9 mg/dL. Test-set clipping counts and class breakdowns were not archived. Clipping could contribute to the recall loss, but these records cannot quantify its contribution for any class. Clipping prevents integer over-

²<https://github.com/mohamedmenasy/GlucoEdge/blob/513e940/docs/superpowers/specs/2026-07-01-litert-conversion-results.md>

Table 6: Artifact and physical-device observations. Timing ends before softmax and class selection. Android sample counts differ by model; iPhone ranges span two rounds of 100 inferences per model.

Measurement	Float	INT8
Artifact size (bytes)	17,352	11,976
Artifact size (KiB)	16.95	11.70
<i>Galaxy S22 Ultra, Android 16, LiteRT 2.1.0</i>		
Mean latency (ms)	0.334 ($n = 100$)	0.484 ($n = 21$)
p95 latency (ms)	0.540	0.665
Golden vectors passed	20/20	20/20
<i>iPhone 18 Pro Max, iOS 27.0.1, LiteRT 2.3.0</i>		
Mean latency (ms)	0.062–0.073	0.101–0.110
p95 latency (ms)	0.103–0.134	0.178–0.182
Golden vectors passed	20/20	20/20

flow but cannot recover information outside the representable range.

All three Android instrumentation tests passed on the physical device. The `CompiledModel` path loaded both artifact hashes, matched float logits within 10^{-5} on all 20 windows, and matched INT8 dequantized outputs exactly. All four iOS parity tests passed on the iPhone with the same result: the artifact hashes, the class-name order, float logits within 10^{-5} , and bit-exact INT8 outputs. The iPhone run used the half-to-even input rounding described above, which the July Android run predates. Python, Kotlin, and Swift therefore agreed on the tested vectors. Model outputs may still differ on untested inputs, although input quantization now follows the same rounding rule on both sides. These tests also cannot establish whether calibration caused the predictive loss.

8 Discussion

8.1 Minority-class performance and deployment cost

INT8 produced a smaller model file and lower recall for every directional class. Integer arithmetic often reduces memory and can improve latency for compute-heavy networks [20]; here the network is only 2,909 parameters, so fixed invocation and data-movement costs may dominate the measured path. The iPhone’s back-to-back benchmark supports this: the same calls ran 20–30 times faster when issued consecutively. Mobile acceleration studies and device benchmarks also show why performance must be measured for a particular model, runtime, and device [17–19]. The observed storage saving was a 5.25-KiB file reduction.

The choice of metric changes how this trade-off appears. The stable class receives approximately three quarters of the weight in accuracy and one fifth in macro recall. Averaging only the directional recalls makes the 14.82-point loss clear. Float better preserves directional performance in these results, which is why GLUCOEDGE uses it by default. This choice applies to the measured

models and does not rank quantization methods in general.

The 5.25-KiB saving concerns model storage. We did not measure whether it reduced application size or working memory. The phone ran both models at the reported latencies. The uncontrolled workloads, and on Android the unequal sample counts, prevent a formal speed comparison. Across the two phones, which differ in hardware, operating system, and LiteRT version, INT8 was never the faster model. Calibration preparation time was also not measured. The results therefore cannot tell us how changing calibration coverage would affect either preparation cost or runtime cost.

8.2 What the calibration evidence identifies

The 200 calibration windows were not the small representative subset of training or validation data that official guidance recommends for estimating tensor ranges [22, 33]. They cover one participant’s 17.5 hours, and the input ceiling matches their maximum, below readings found in about a quarter of training windows. Prior PTQ work identifies clipping and range estimation as error sources [23], optimizes weight rounding [24], and improves quantization with small calibration sets [25]. These studies suggest explanations to test, but do not identify the main source of error in this CGM model.

Broader range coverage is not guaranteed to improve recall. For a fixed integer code range, increasing the represented interval increases the scale s and thus the spacing between adjacent representable inputs. Less clipping therefore comes at the cost of coarser resolution. The future-derived class depends on a difference between two readings; a high absolute glucose value does not by itself identify a rapid rise or fall. The aggregate saturation figure does not tell us whether clipping occurs more often in any particular directional class.

A follow-up experiment could compare random calibration selection with selection designed to cover the input range, while keeping model weights, quantizer, calibration budget, and participant allocation fixed. Selection must use development inputs without tuning to test labels or test ranges. Passing quantized and then dequantized inputs through the float model would measure input-only loss for comparison with full INT8 loss. The remaining difference could reflect internal quantization and its interaction with input error. Before this comparison, the validation-row targets must be removed from the test frame. Evaluation should then report each directional recall, paired participant-level uncertainty, and clipped-input fractions by class. Device measurements would need identical workloads and controlled repeated runs. These experiments remain to be done.

The parity tests check agreement for particular inputs and artifact hashes. Unit tests cover exact rounding ties and out-of-range values in the Kotlin and Swift quantiz-

ers, but model execution has been compared only on the tested vectors. Runtime agreement alone cannot separate calibration error from other quantization effects. It also does not validate the labels, cohort split, probability calibration, clinical safety, or usefulness to a person with diabetes.

9 Limitations

The results should be read with the following limitations in mind.

1. The deployed checkpoint comes from one seed. Training and implementation choices can affect benchmark results [15], but we did not measure across-seed variability or calculate confidence intervals. We also did not evaluate probability calibration, event-level performance, or input-dependent temporal baselines beyond the `constant-stable` comparator.
2. Stride-one windows are overlapping and statistically dependent, so their totals are not independent event counts [13]. Because GlucoBench interpolates before window construction, only 56.1% of test windows have all inputs and the target endpoint on original readings, and 76.5% contain no gap-filled value. Interpolated inputs can depend on later readings, so this retrospective preprocessing may use information that would be unavailable in real time. Results are not stratified by interpolation status.
3. GlucoBench adds prefix context to validation and in-distribution test frames so sequence models can form initial histories. The current GLUCOEDGE dataset builder lets every prefix row start a window and does not mask context-only targets. As a result, 17,266 test windows (13.6%) score targets in validation rows, and 19,402 validation windows (35.1%) repeat training windows (Table 3). Using earlier readings as inputs can be legitimate. No test target lies in a training row, but scoring context-only targets blurs the temporal boundary between validation and test. A corrected evaluation must mask context-only targets, verify disjoint scored participant/segment/time keys, and report temporal and held-out-participant components separately.
4. The aggregate test frame mixes temporal holdouts with participants held out by GlucoBench. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12], and no cross-dataset validation was performed. We therefore cannot assess transfer across cohorts and collection pipelines [4]. No participant-stratified, demographic, daytime/nighttime, or subgroup results are available.

5. The five labels are deterministic transformations of a future CGM point, not expert annotations. Their thresholds simplify heterogeneous commercial trend-arrow conventions. The CGM-only model omits meal, insulin, activity, illness, and other covariates.
6. The INT8 calibration set consists of the first 200 validation windows, all from one participant’s 17.5 hours, not a randomized or stratified sample. The observed saturation is therefore specific to this conversion protocol. It should not be generalized to INT8. Without comparisons of calibration samplers or budgets, we cannot estimate the effect of coverage, a dose-response relation, or possible recall recovery.
7. The phone latency observations use two devices that differ in hardware, operating system, LiteRT version (2.1.0 and 2.3.0), and timer. Android sample counts are unequal. The iPhone counts are equal but cover only two rounds, recorded with a debug build of the application code. Warm-up, temperature, and processor frequency were not controlled, and repeated runs were not analyzed statistically. Calibration time, model-load time, energy, and memory use were not measured. The internal timing excludes softmax and class selection.
8. The application replays synthetic or locally extracted historical traces. It does not ingest a live sensor stream and has not undergone prospective, external, usability, or clinical validation.

10 Reproducibility and Ethics

10.1 Code, artifacts, and computational record

The repository contains the training, conversion, benchmark, Android, iOS, and test code. The bundled artifacts were generated from repository commit 361148a with seed 0. Their SHA-256 digests are `eb96c7e...08e6` (float) and `0dc35387...ea29` (INT8); full hashes are stored beside the assets and asserted by instrumentation tests. Python unit tests cover label boundaries, segment-aware windows, model shape, conversion helpers, synthetic replay behavior, and the provenance audit’s split bookkeeping. Android unit tests cover replay, buffering, quantization, latency aggregation, and view-model behavior. A merged-manifest check enforces the absence of Internet permission. The iOS test suite covers the same units and the golden-vector parity. Continuous integration runs it on a simulator, together with the source scan for networking APIs.

The conversion requirements record LiteRT Torch 0.9.1, TorchAO 0.17.0, and Python LiteRT 2.1.5. They also document PyTorch 2.12.1 and NumPy 2.5.0 in the

historical environment, although the top-level requirements pinned older versions. These records do not constitute a complete environment lock. PyTorch seed 0 and the formatter’s split seed 0 control separate randomization steps; neither guarantees bitwise retraining across platforms. The artifact hashes identify the exact models used for replay.

For the September manuscript revision, we reused the July results and calculated descriptive summaries from the rounded tables. We did not retrain models, rerun evaluation on the research data, or collect new phone timings. Machine-readable run reports and the training checkpoint can be regenerated locally, but are not versioned paper supplements. A fresh run may produce different numbers from those recorded here.

A subsequent provenance audit reran only the GlucoBench formatter and window construction; it involved no training, model evaluation, or held-out glucose values.

The October revision adds the iOS client and its device measurements, recorded on 10 October 2026 with the unchanged artifacts (same SHA-256 hashes). It involved no retraining, conversion, or evaluation on research data. Its aggregate output, including the GlucoBench commit and a SHA-256 hash of the Weinstock data file, is versioned as `paper/results/split_audit.json`.

10.2 Data availability and research-use scope

The CGM data are public research data accessed through GlucoBench under the source dataset’s reuse terms [1]. They are not committed to the GLUCOEDGE repository or the distributed Android build. The committed replay trace is synthetic. This work makes no diagnosis, monitoring, treatment, or dosing claim. Its outputs must not be used for medical decisions. The application displays this restriction in the primary interface.

Code, bundled model files, and synthetic golden vectors are available at <https://github.com/mohamedmenasy/GlucoEdge>; source CGM access is described by GlucoBench [1]. The work uses secondary research data and the prototype has not been studied with newly recruited participants. We do not claim an institutional ethics determination for this work.

10.3 AI assistance and author declarations

OpenAI Codex assisted with language editing during the September manuscript revision, checked sources, and compared claims with repository code and recorded results. Claude Code (Anthropic) wrote the provenance audit script and the Kotlin rounding fix, implemented the iOS client, and drafted the corresponding manuscript edits. The draft requires author review. Funding, competing interests, and CRediT author contributions remain to be confirmed by the author before submission.

11 Conclusion

GLUCOEDGE follows a compact five-class CGM trend model from segment-aware window construction to offline execution on Android and iOS. The implementation documents fixed-shape conversion, integer input handling, artifact provenance, and cross-runtime parity. Under the recorded calibration protocol, INT8 saved 5.25 KiB while reducing five-class macro recall by 9.25 percentage points and mean directional recall by 14.82 points. Its higher accuracy comes from improved stable-class recall and that class’s large share of the test windows. The INT8 input range was derived from calibration windows drawn from a single participant, which makes calibration coverage worth investigating, but this comparison cannot establish whether broader coverage would recover recall or how it would affect preparation and deployment cost. Neither phone showed an INT8 latency advantage, including the iPhone, where both models had equal sample counts. These measurements were not controlled benchmarks.

These results provide a baseline for a controlled calibration-coverage study. That study needs corrected target boundaries, comparisons of calibration selections, repeated seeds, and separate participant-level evaluation. External validation would follow. This work remains a research prototype.

References

- [1] Renat Sergazinov, Elizabeth Chun, Valeriya Rogovchenko, Nathaniel Fernandes, Nicholas Kasman, and Irina Gaynanova. GlucoBench: Curated list of continuous glucose monitoring datasets with prediction benchmarks. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=cUSNs8nGaV>.
- [2] Kui Liu, Linyi Li, Yifei Ma, Jun Jiang, Zhenhua Liu, Zichen Ye, Shuang Liu, Chen Pu, Changsheng Chen, and Yi Wan. Machine learning models for blood glucose level prediction in patients with diabetes mellitus: Systematic review and network meta-analysis. *JMIR Medical Informatics*, 11:e47833, 2023. doi: 10.2196/47833.
- [3] Andrea Facchinetti, Giovanni Sparacino, Emanuele Trifoglio, and Claudio Cobelli. A new index to optimally design and compare continuous glucose monitoring glucose prediction algorithms. *Diabetes Technology & Therapeutics*, 13(2):111–119, 2011. doi: 10.1089/dia.2010.0151.
- [4] Sarala Ghimire, Turgay Celik, Martin Gerdes, and Christian W. Omlin. Deep learning for blood glucose level prediction: How well do models generalize across different data sets? *PLOS ONE*, 19(9):e0310801, 2024. doi: 10.1371/journal.pone.0310801.
- [5] Jeremy Pettus and Steven V. Edelman. Recommendations for using real-time continuous glucose monitoring (rtCGM) data for insulin adjustments in type 1 diabetes. *Journal of Diabetes Science and Technology*, 11(1):138–147, 2017. doi: 10.1177/1932296816663747.

- [6] Melanie Rodacki, Luis Eduardo Calliari, Antonio Carlos Ramalho, et al. Using trend arrows in continuous glucose monitoring systems for insulin adjustment in clinical practice: Brazilian diabetes society position statement. *Diabetology & Metabolic Syndrome*, 13(1):2, 2021. doi: 10.1186/s13098-020-00607-2.
- [7] Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.
- [8] Jaques Reifman, Srinivasan Rajaraman, Andrei Gribok, and W. Kenneth Ward. Predictive monitoring for improved management of glucose levels. *Journal of Diabetes Science and Technology*, 1(4):478–486, 2007. doi: 10.1177/193229680700100405.
- [9] Sadegh Mirshekarian, Hui Shen, Razvan Bunescu, and Cindy Marling. LSTMs and neural attention models for blood glucose prediction: Comparative experiments on real and synthetic data. In *41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 706–712, 2019. doi: 10.1109/EMBC.2019.8856940.
- [10] John Martinsson, Alexander Schliep, Björn Eliasson, and Olof Mogren. Blood glucose prediction with variance estimation using recurrent neural networks. *Journal of Healthcare Informatics Research*, 4(1):1–18, 2020. doi: 10.1007/s41666-019-00059-y.
- [11] Kezhi Li, John Daniels, Chengyuan Liu, Pau Herrero, and Pantelis Georgiou. Convolutional recurrent neural networks for glucose prediction. *IEEE Journal of Biomedical and Health Informatics*, 24(2):603–613, 2020. doi: 10.1109/JBHI.2019.2908488.
- [12] Ruth S. Weinstock, Stephanie N. DuBose, Richard M. Bergenstal, et al. Risk factors associated with severe hypoglycemia in older adults with type 1 diabetes. *Diabetes Care*, 39(4):603–610, 2016. doi: 10.2337/dc15-1426.
- [13] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018. doi: 10.1016/j.csda.2017.11.003.
- [14] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.
- [15] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3c3fb185989f858a1d9f5c1eb-Abstract.html.
- [16] Serkan Kiranyaz, Onur Avci, Osama Abdeljaber, Turker Ince, Moncef Gabbouj, and Daniel J. Inman. 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151:107398, 2021. doi: 10.1016/j.ymssp.2020.107398.
- [17] Nicholas D. Lane, Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro, and Fahim Kawsar. DeepX: A software accelerator for low-power deep learning inference on mobile devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, pages 1–12, 2016. doi: 10.1109/IPSN.2016.7460664.
- [18] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. AI benchmark: All about deep learning on smartphones in 2019. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, pages 3617–3635, 2019. doi: 10.1109/ICCVW.2019.00447.
- [19] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, et al. MLPerf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html.
- [20] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.
- [21] PyTorch. PT2E quantization, 2026. URL https://docs.pytorch.org/ao/stable/pt2e_quantization/index.html. Accessed 2026-09-19.
- [22] Google AI Edge. Representative dataset for model quantization. Google AI for Developers documentation, 2026. URL <https://developers.google.com/edge/api/tflite/python/tf/lite/RepresentativeDataset>. Accessed 2026-09-19.
- [23] Ron Banner, Yury Nahshan, and Daniel Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html>.
- [24] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7197–7206, 2020. URL <https://proceedings.mlr.press/v119/nagel20a.html>.
- [25] Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. Accurate post training quantization with small calibration sets. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4466–4475, 2021. URL <https://proceedings.mlr.press/v139/hubara21a.html>.
- [26] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [27] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information*

- Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [28] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
 - [29] NumPy Developers. `numpy.round`. NumPy reference documentation, 2026. URL <https://numpy.org/doc/stable/reference/generated/numpy.round.html>. Accessed 2026-09-19.
 - [30] JetBrains. `kotlin.math.round`. Kotlin standard library reference, 2026. URL <https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.math/round.html>. Accessed 2026-10-09.
 - [31] Apple. `FloatingPointRoundingRule.toNearestOrEven`. Swift standard library documentation, 2026. URL <https://developer.apple.com/documentation/swift/floatingpointroundingrule/tonearestoreven>. Accessed 2026-10-10.
 - [32] Google AI Edge. Run LiteRT compiled model API on Android with Kotlin. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/next/android_kotlin. Accessed 2026-07-20.
 - [33] Google AI Edge. Post-training quantization with LiteRT. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization. Accessed 2026-09-19.